

COUNTRY CLASH
Videojuego 2D en Unity

Entrega 2 – Proyecto DAM

Alumna: Catalina Angue Mba
Ciclo: Desarrollo de Aplicaciones
Multiplataforma (DAM)

Profesor:Manuel Prieto Moriche

Fecha: 8 de mayo de 2026

Introducción	3
Objetivo.....	3
Descripción general del proyecto.....	3
Tecnologías empleadas	4
Desarrollo del proyecto (diseño y estructura)....	5
Desarrollo del proyecto (funcionalidades y etapas)	6
Base de datos (diseño y estructura).....	8
Modelo Entidad-Relación	9
Entidades.....	10
Relaciones	11
Pruebas, errores y soluciones	13
Conclusiones	14
Bibliografía	15
Licencia.....	15

Introducción

Objetivo

El objetivo de este proyecto es desarrollar un videojuego 2D en Unity en el que el jugador compite contra bots que representan distintos países.

La idea principal es que cada participante recoja únicamente los objetos de su país para conseguir la mayor puntuación posible.

Durante el desarrollo he intentado centrarme en que la lógica del juego funcione correctamente, especialmente en la parte de la competición entre jugador y bots.

Descripción general del proyecto

Country Clash es un videojuego 2D en el que el jugador representa a un país y compite contra otros países controlados por bots.

A lo largo de la partida van apareciendo objetos por el mapa, cada uno asociado a un país. Tanto el jugador como los bots deben recoger solo los objetos que les corresponden.

La partida tiene un tiempo limitado y, cuando termina, se calcula la puntuación y se muestra un ranking con los resultados ordenados.

El proyecto está planteado de forma que se pueda ampliar en el futuro, por ejemplo mejorando la inteligencia artificial, el diseño visual o añadiendo más modos de juego.

Tecnologías empleadas

- **Unity Engine:** utilizado como motor principal para el desarrollo del juego, gestión de escenas y físicas.
- **C#:** lenguaje utilizado para programar toda la lógica (movimiento, bots, puntuación, etc.).

- **TextMeshPro:** para mostrar textos en pantalla como puntuaciones, tiempo o resultados.
- **Visual Studio:** entorno utilizado para programar y depurar.
- **Sistema de escenas de Unity:** para organizar el flujo del juego (menú, juego, ranking).
- **Tilemaps de Unity:** El sistema de Tilemaps se ha utilizado para construir el escenario del juego. El mapa está dividido en diferentes capas, tilemap de suelo y tilemap de obstáculos.
- **Animator y Animator Override Controller:** se ha utilizado para controlar las animaciones de personajes y bots.
- **MySQL (Railway cloud database):** utilizado como sistema de base de datos en la nube para almacenar las puntuaciones

de las partidas y permitir un ranking global persistente.

- **PHP (API REST):** utilizado para crear una API intermedia que permite la comunicación entre Unity y la base de datos MySQL.

- **XAMPP (entorno local de desarrollo):** utilizado para ejecutar y probar la API en entorno local durante el desarrollo del proyecto.

Desarrollo del proyecto (diseño y estructura)

El desarrollo del proyecto se ha realizado de forma progresiva, construyendo el videojuego por fases desde la base hasta la implementación de sistemas más complejos.

Etapas 1 : Estructura inicial del proyecto

Se crearon las escenas principales del juego:

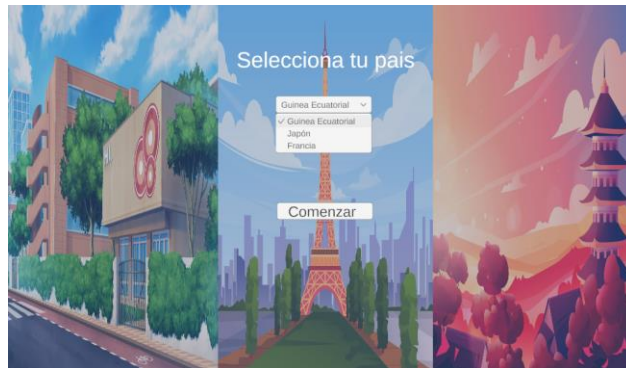
En esta fase también se definió la arquitectura general del proyecto en Unity.

Menu principal



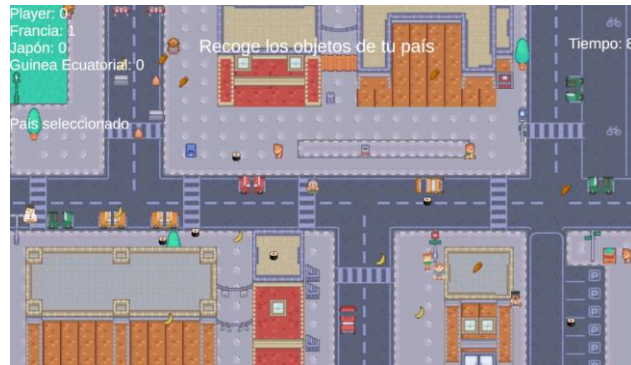
En esta pantalla se muestra el menú principal del videojuego Country Clash. Desde aquí el jugador puede iniciar una nueva partida, acceder al ranking global o salir del juego. Esta escena actúa como punto de entrada al sistema y gestiona la navegación entre las distintas partes del proyecto mediante el sistema de escenas de Unity.

Selección de país



En esta escena el jugador selecciona el país con el que va a jugar la partida. Esta elección determina el personaje controlado por el usuario y el conjunto de objetos que deberá recoger durante el juego. La selección también influye en la lógica de puntuación y en la identificación del jugador dentro del sistema de ranking.

Escena de juego



Esta es la escena principal del juego, donde se desarrolla la partida. El jugador controla un personaje que se mueve por el mapa mientras compite contra bots controlados por inteligencia artificial básica.

Durante la partida aparecen objetos asociados a distintos países, y cada personaje debe recoger únicamente aquellos que correspondan a su país. Cada recogida correcta incrementa la puntuación del jugador o bot.

En la parte superior se muestra la interfaz de usuario con la puntuación en tiempo real y el temporizador de la partida.

Panel previous al ranking



En esta pantalla se muestra el resultado final de la partida una vez el tiempo ha terminado. Se genera un ranking ordenando a los participantes según la puntuación obtenida.

Además, se indica el ganador de la partida y se muestra un mensaje de resultado (victoria o derrota en el caso del jugador). Este ranking se genera dinámicamente durante la ejecución del juego.

Escena de ranking



En esta pantalla se muestra el ranking global del juego obtenido desde una base de datos MySQL alojada en la nube. Los datos se recuperan mediante una API desarrollada en PHP que conecta Unity con la base de datos.

Esto permite almacenar y consultar puntuaciones reales de partidas anteriores, proporcionando persistencia de datos y un sistema de clasificación global.

Etapas 2 : Movimiento y lógica básica

Se implementó el control del jugador mediante teclado, así como la base del sistema de escenas y navegación dentro del juego.

Etapas 3 : Sistema de spawn

Se desarrolló el sistema encargado de generar:

- jugador
- bots
- objetos coleccionables

Los objetos aparecen en posiciones aleatorias dentro del mapa.

Etapas 4 : Inteligencia artificial de los bots

Se implementó una IA básica para los bots, que les permite:

- detectar objetos de su país
- moverse hacia ellos automáticamente
- competir con el jugador

Etapas 5 : Sistema de colisiones y Tilemaps

Se rediseñó el escenario utilizando Tilemaps:

- Tilemap de suelo (solo visual)
- Tilemap de obstáculos (con colliders)

Se solucionaron problemas de atravesamiento de objetos y colisiones incorrectas.

Etapas 6 : Animaciones y mejora visual

Se implementó el sistema de animaciones utilizando:

- Animator base
- Animator Override Controller

Se corrigieron problemas de estados bloqueados y animaciones incorrectas.

Etapas 7 : Sistema de puntuación y ranking local

Se desarrolló el sistema de puntuación en tiempo real y el ranking final dentro de la partida.

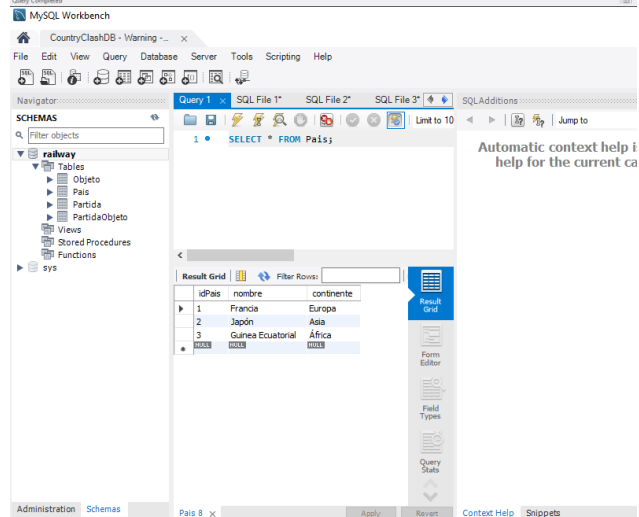
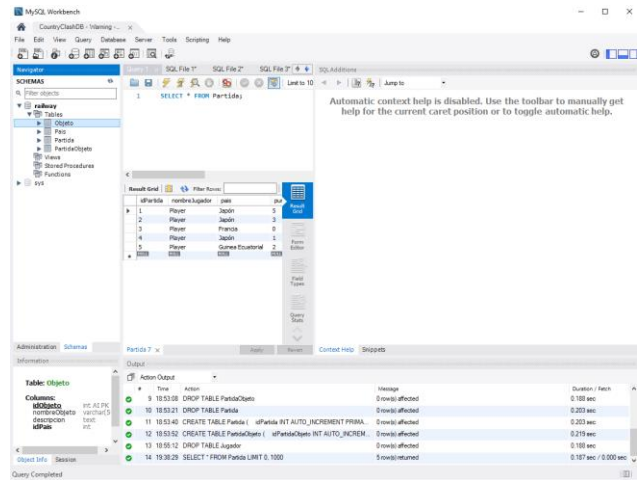
Etapas 8 : Base de datos y ranking online

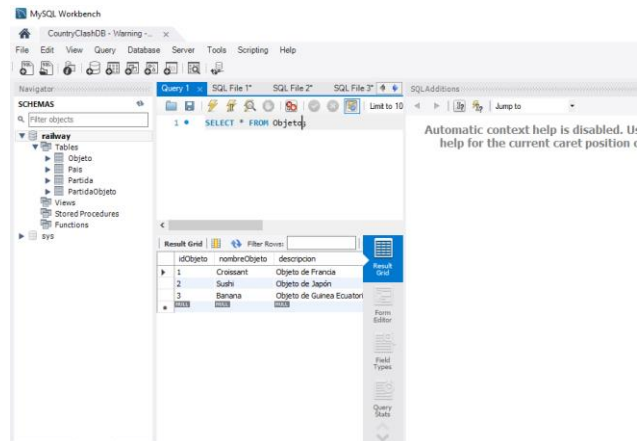
Se implementó una arquitectura cliente servidor:Unity, API PHP, MySQL (Railway)

Esta fase permitió:

- guardar puntuaciones reales
- consultar ranking global

- persistencia de datos online





En estas imágenes se muestra la estructura de la base de datos utilizada en el proyecto, alojada en un servicio cloud (Railway). La base de datos almacena información de partidas, países y objetos.

Este sistema permite guardar y recuperar información de las partidas de forma persistente, facilitando la implementación del ranking online y futuras ampliaciones del proyecto.

Desarrollo del proyecto (funcionalidades y etapas)

El juego comienza en el menú principal, donde el jugador puede empezar una partida, ver el ranking o salir.

Antes de jugar, se selecciona un país. Esto afecta tanto al comportamiento como a los objetos que se deben recoger.

El sistema de spawn se encarga de crear el jugador, crear los bots y generar los objetos coleccionables. Los objetos aparecen aleatoriamente dentro de un área definida desde el Inspector. También se puede configurar la cantidad de objetos por país .

Durante la partida:

- El jugador se mueve por el escenario con el teclado
- Aparecen objetos en posiciones aleatorias
- Cada personaje (jugador o bot) tiene un país asignado
- Solo puede recoger objetos de su país
- Si recoge uno correcto, suma puntos

Los bots funcionan buscando objetos cercanos de su país y moviéndose hacia ellos automáticamente.

Uno de los problemas al principio fue que los bots recogían objetos incorrectos, pero se solucionó añadiendo una comprobación del país antes de sumar puntos.

La partida dura un tiempo limitado. Cuando se acaba:

- Se muestra un panel con el resultado (ganar, perder o empate)
- Se calcula el ranking ordenado
- Se indica el ganador

El sistema de animaciones utiliza idle y walk se reorganizó usando un Animator base y Animator Override Controllers.

Base de datos (diseño y estructura)

El sistema de datos del proyecto está diseñado mediante un modelo relacional que organiza la información del juego en

diferentes entidades: jugadores, partidas, países y objetos.

Inicialmente el proyecto estaba planteado para utilizar SQLite como sistema de almacenamiento local. Sin embargo, durante el desarrollo se decidió migrar hacia una solución basada en MySQL alojada en la nube mediante Railway.

Este cambio permitió implementar persistencia online de puntuaciones y un sistema de ranking global conectado directamente con Unity mediante una API desarrollada en PHP.

La arquitectura final implementada fue:

Unity , API PHP , MySQL Cloud

El uso de una API intermedia evita conectar Unity directamente con la base de datos, mejorando la organización del proyecto y la seguridad del sistema.

La base de datos almacena información de: jugadores, países, partidas, objetos y puntuaciones .

El sistema permite registrar puntuaciones reales obtenidas durante las partidas y mostrarlas posteriormente en el ranking online del juego.

Durante el desarrollo del proyecto se ha adaptado el modelo de base de datos inicialmente planteado en la fase de diseño.

Aunque el modelo original incluía varias entidades (Jugador, Partida, País, Objeto y PartidaObjeto), la implementación final se ha simplificado para ajustarse al alcance real del proyecto y al tiempo disponible.

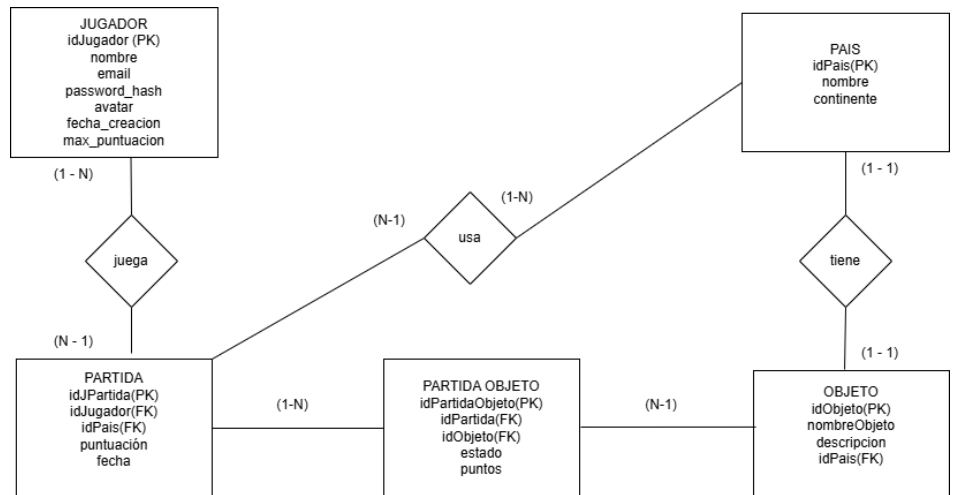
La base de datos finalmente implementada en MySQL (Railway) se centra en el almacenamiento de puntuaciones de partidas para el ranking global online.

Esto ha permitido priorizar la funcionalidad principal del juego: la competición y el sistema de clasificación de jugadores.

Las tablas utilizadas finalmente son: Jugador y Partida .

Y la información de objetos y países se gestiona dentro de la lógica del juego en Unity, ya que no requieren persistencia externa.

Este enfoque simplificado ha permitido mantener una arquitectura funcional, estable y coherente con el objetivo principal del proyecto.



Modelo Entidad-Relación

Para almacenar la información del juego se ha diseñado un modelo relacional sencillo que permite gestionar jugadores, partidas, países y objetos.

Entidades

Jugador

- PK idJugador
- nombre
- email
- password_hash

- avatar
- fecha_creacion
- max_puntuacion

Partida

- PK idPartida
- FK idJugador
- FK idPais
- puntuacion
- fecha

País

- PK idPais
- nombre
- continente

Objeto

- PK idObjeto
- nombreObjeto
- descripcion
- FK idPais

PartidaObjeto

- PK idPartidaObjeto
- FK idPartida
- FK idObjeto
- estado
- puntos

Relaciones

- Jugador – Partida (1 : N)
Un jugador puede participar en múltiples partidas, pero cada partida pertenece a un único jugador.
- País – Partida (1 : N)
Un país puede ser seleccionado en muchas partidas, pero cada partida se juega con un solo país.
- País – Objeto (1 : N)
Un país puede tener varios objetos asociados, pero cada objeto pertenece a un único país.
- Partida – Objeto (N : N)
Una partida puede incluir múltiples objetos recogidos, y un mismo objeto puede aparecer en distintas partidas.

Esta relación se resuelve mediante la entidad intermedia PartidaObjeto.

- Partida – PartidaObjeto (1 : N)
- Objeto – PartidaObjeto (1 : N)

La entidad PartidaObjeto permite registrar qué objetos se han recogido en cada partida y los puntos obtenidos.

El modelo definido en la entrega anterior sigue siendo válido.

Las mejoras realizadas (bots, ranking, resultado final) no requieren cambios en la base de datos, ya que se gestionan en la lógica del juego.

Correcciones, mejoras y ampliaciones realizadas(planteadas por el tutor, como por iniciativa propia)

Siguiendo las indicaciones del tutor, se han realizado las siguientes mejoras en la memoria y en el desarrollo del proyecto:

- Se han añadido capturas explicativas del videojuego para mejorar la comprensión visual del proyecto.

- Se ha reorganizado el apartado de desarrollo, separando claramente las fases de construcción del proyecto del funcionamiento del juego.
- Se ha ampliado el apartado de conclusiones, incluyendo aprendizajes, problemas encontrados y posibles mejoras futuras.
- Se ha priorizado el correcto funcionamiento del juego como requisito principal de la entrega final.

Mejora visual del proyecto

Uno de los principales cambios realizados fue sustituir las figuras geométricas simples por sprites más trabajados. Esto mejoro la apariencia visual, la diferenciación entre personajes y la claridad del juego . Tambien se cambio el objeto de coleccion de Guinea Ecuatorial, el cual al principio estaba pensado en el árbol representativo del país, pero al no encontrar un sprite concreto del arbol, decidí que los objeto representativo de cada país seria el alimento.

Reorganización de Tilemaps

El escenario fue reconstruido utilizando un tilemap decorativo y un tilemap de obstáculos, con esto se mejoro la

organización del mapa, las colisiones y movimiento de personajes.

Sistema de animaciones reorganizado

Al principio hubieron muchos errores relacionados con animators distintos, estados bloqueados, transiciones incorrectas.

La solución fue reorganizar completamente el sistema usando animator base y animator override controllers.

Mejora del sistema de colisiones

Fueron corregidos los errores donde los personajes atravesaban objetos y los bots recogían objetos a distancia, ahora la recogida solo ocurre mediante colisión real.

Mejor organización del código

Los scripts se simplificaron y reorganizaron para facilitar la depuración y las futuras modificaciones.

Implementación de arquitectura online

Se sustituyó el sistema de almacenamiento local (SQLite) por una arquitectura online basada en Unity, API PHP, MySQL (Railway).

Esto permitió implementar un sistema de ranking global persistente.

Pruebas, errores y soluciones

Durante el desarrollo del proyecto se han detectado y solucionado diversos errores técnicos:

- Los bots recogían objetos de otros países, lo que rompía la lógica del juego.
Se solucionó validando el país del objeto antes de sumar puntuación.
- Se producían duplicaciones de personajes en escena.
Se corrigió controlando la instanciación en el sistema de spawn.
- Errores en la activación del panel de fin de partida.
Se solucionaron verificando correctamente las referencias en el GameManager.
- Problemas en la actualización del ranking.
Se corrigió mediante la ordenación dinámica de puntuaciones al finalizar la partida.
- Algunos bots no tenían Animator asignado.
Se revisaron prefabs y se reorganizó el sistema de animaciones.

- Personajes en estado de salto permanente.
Se eliminó el sistema de salto y se simplificaron las animaciones.
- Personajes atravesaban obstáculos.
Se solucionó reestructurando los Tilemaps y colisiones.
- Bots se detenían cuando no había objetos disponibles.
Se añadió un sistema de movimiento aleatorio.
- Incompatibilidad entre sprites de jugador y bots.
Se resolvió usando Animator Override Controllers.

Conclusiones

El desarrollo de Country Clash me ha permitido aplicar y consolidar conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Multiplataforma, especialmente en áreas como programación en C#, desarrollo de videojuegos en Unity y diseño de sistemas interactivos.

Durante el proyecto se ha trabajado de forma progresiva, comenzando por una base sencilla y añadiendo funcionalidades como movimiento del jugador, inteligencia

artificial básica para los bots, sistema de colisiones mediante Tilemaps y animaciones.

Uno de los aspectos más importantes del proyecto ha sido la evolución hacia una arquitectura más completa, incorporando una base de datos MySQL alojada en la nube mediante Railway. Para la comunicación entre Unity y la base de datos se ha desarrollado una API en PHP, lo que ha permitido implementar un sistema de ranking online funcional.

Este cambio ha supuesto una mejora significativa en la estructura del proyecto, pasando de un sistema local a una arquitectura cliente-servidor más cercana a aplicaciones reales.

Durante el desarrollo se han encontrado diversos problemas técnicos relacionados con animaciones, colisiones, instanciación de objetos y comunicación entre sistemas. La resolución de estos errores ha permitido mejorar la estabilidad del proyecto y comprender mejor el funcionamiento interno de Unity y de la arquitectura backend.

El proyecto ha evolucionado considerablemente desde su idea inicial, incorporando múltiples sistemas como IA básica, gestión de escenas, puntuaciones y persistencia de datos.

Como posibles mejoras futuras se podrían añadir:

- sistema de login y registro de usuarios
- guardado completo del historial de partidas
- mejoras en la inteligencia artificial
- efectos de sonido y música
- nuevos mapas y países

En conclusión, este proyecto ha permitido desarrollar una aplicación completa, funcional y cercana a una arquitectura real utilizada en videojuegos modernos.

Bibliografía

- Unity Technologies. (2024). Unity Documentation. Recuperado de <https://docs.unity.com>
- Microsoft. (2024). C# Documentation. Recuperado de <https://learn.microsoft.com/dotnet/csharp>
- Unity Technologies. (2024). TextMeshPro Documentation. Recuperado de

<https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest>

- Unity Learn. (2024). Tutorials and educational resources for Unity 2D.
Recuperado de
<https://learn.unity.com>

Licencia

Este proyecto se distribuye bajo una licencia de uso educativo.

Su código y contenido pueden ser utilizados con fines académicos, no comerciales.